Experiment: -1

Aim: To perform version control operations using Git and understand various Git commands for repository, file, branch, and commit management.

This experiment maps to LO2.

LO2: Use Git effectively for version control, source code management and collaborative team management.

Theory:

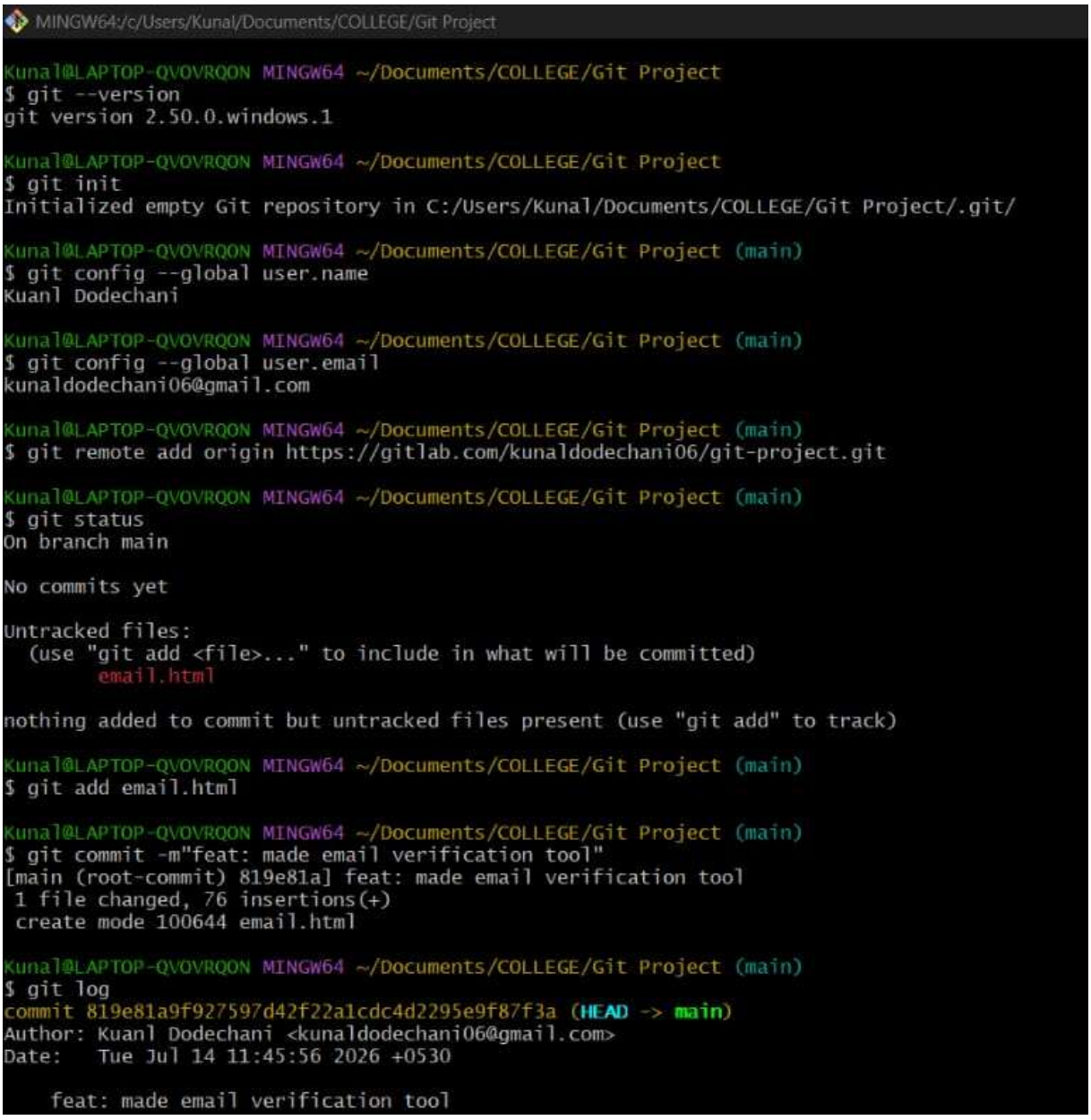
Git is a distributed version control system used to track changes in files and source code. It allows users to create repositories, maintain different versions of a project, work with branches, and view the history of changes. Git provides commands for adding, committing, comparing, merging, and managing files efficiently. In this experiment, 25 basic Git operations are performed to understand version control.

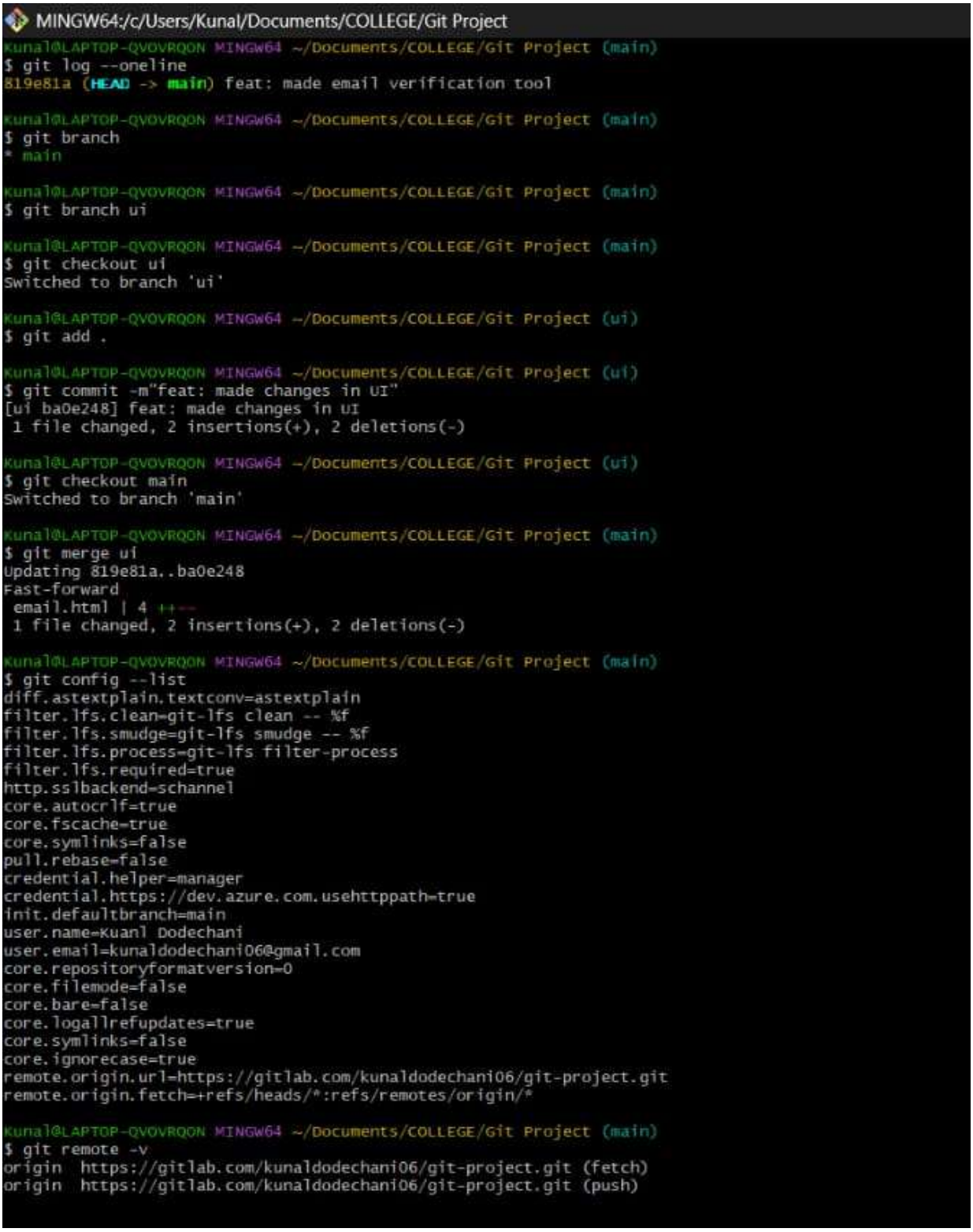
Commands:

Command	Short Description
git --version	Displays the installed Git version.
git init	Initializes a new Git repository in the current folder.
git config --global user.name	Displays the globally configured Git username.
git config --global user.email	Displays the globally configured Git email address.
git remote add origin <repository-url>	Connects the local repository to a remote repository (GitLab/GitHub).
git status	Shows the current status of the repository, including staged, modified, and untracked files.
git add email.html	Adds the file email.html to the staging area.

git add	Stages the current changes (used before committing).
git commit -m "feat: made email verification tool"	Creates a commit with the specified message.
git log	Displays the complete commit history.
git log --oneline	Displays a compact version of the commit history with short commit IDs.
git branch	Lists all local branches and indicates the current branch.
git branch ui	Creates a new branch named ui.
git checkout ui	Switches from the current branch to the ui branch.
git commit -m "feat: made changes in UI"	Saves the changes made in the ui branch.
git checkout main	Switches back to the main branch.
git merge ui	Merges the ui branch into the current (main) branch.
git config *(used to display configuration)*	Displays the current Git configuration settings. *(Usually git config --list is used.)*
git remote -v	Displays the configured remote repositories along with their fetch and push URLs.

Output:


MINGW64:/c/Users/Kunal/Documents/COLLEGE/Git Project
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project
$ git --version
git version 2.50.0.windows.1
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project
$ git init
Initialized empty Git repository in C:/Users/Kunal/Documents/COLLEGE/Git Project/.git/
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git config --global user.name
Kuanl Dodechani
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git config --global user.email
kunaldodechani06@gmail.com
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git remote add origin https://gitlab.com/kunaldodechani06/git-project.git
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git status
On branch main
No commits yet
Untracked files:
(use "git add <file>..." to include in what will be committed)
email.html
nothing added to commit but untracked files present (use "git add" to track)
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git add email.html
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git commit -m"feat: made email verification tool"
[main (root-commit) 819e81a] feat: made email verification tool
1 file changed, 76 insertions(+)
create mode 100644 email.html
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git log
commit 819e81a9f927597d42f22a1cdc4d2295e9f87f3a (HEAD -> main)
Author: Kuanl Dodechani <kunaldodechani06@gmail.com>
Date: Tue Jul 14 11:45:56 2026 +0530
feat: made email verification tool


MINGW64:/c/Users/Kunal/Documents/COLLEGE/Git Project
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git log --oneline
819e81a (HEAD -> main) feat: made email verification tool
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git branch
* main
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git branch ui
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git checkout ui
Switched to branch 'ui'
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (ui)
$ git add .
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (ui)
$ git commit -m"feat: made changes in UI"
[ui ba0e248] feat: made changes in UI
1 file changed, 2 insertions(+), 2 deletions(-)
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (ui)
$ git checkout main
Switched to branch 'main'
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git merge ui
Updating 819e81a..ba0e248
Fast-forward
email.html | 4 ++--
1 file changed, 2 insertions(+), 2 deletions(-)
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=main
user.name=Kuanl Dodechani
user.email=kunaldodechani06@gmail.com
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
remote.origin.url=https://gitlab.com/kunaldodechani06/git-project.git
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
Kunal@LAPTOP-QVOVRQON MINGW64 ~/Documents/COLLEGE/Git Project (main)
$ git remote -v
origin https://gitlab.com/kunaldodechani06/git-project.git (fetch)
origin https://gitlab.com/kunaldodechani06/git-project.git (push)

Conclusion: This experiment helped me understand the basics of Git and version control. I learned how to create a repository, track and commit changes, manage branches, restore previous versions, and work with remote repositories. Overall, Git makes it easy to organize, track, and manage project files efficiently during software development.